COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE 21S3
DATE PERFORMED:	10/15/26	**DATE SUBMITTED:**	10/15/26
NAME:	Busa, Emmanuel A. Joncel Zairho G. Geronimo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **Parallelism and Multithreading**

1. PROCEDURE OUTPUT

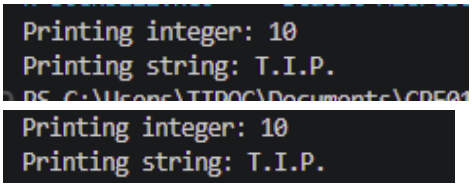
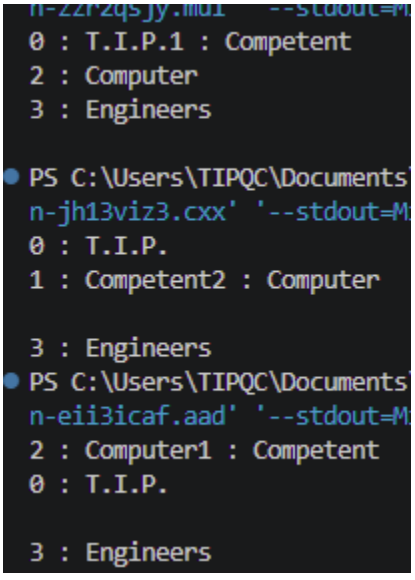
Screenshot	
Analysis	A single thread executes the print function safely and sequentially, using .join() to wait for completion.

Table 13-1. Simple One-Threaded Example

Screenshot	
Analysis	The multithreaded program executes multiple threads at the same time, making the output appear different each time the code runs. This shows that each thread works independently, making the program more

	efficient.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.

Busa: Multithreading makes the program run multiple threads at the same time. Its advantages are faster execution and better CPU usage, while its disadvantages are that it can be harder to manage and may cause errors when threads access the same data.

Geronimo: Multithreading involves creating and using thread objects to carry out delegated sub-tasks independently. Its main advantage is to efficiently utilize CPU resources to handle complex operations. However, its disadvantages include data races and unpredictable behavior if multiple threads access shared memory without synchronization tools.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Busa: Multithreading is useful in applications like games, web servers, and video editing. It helps games run different tasks, web servers handle multiple users, and video editors process videos faster.

Geronimo: Multithreading is needed for maximizing CPU resources and accelerating complex operations. 3use-cases are maintaining UI responsiveness, managing server requests, and speeding up data processing

3. Differentiate between parallelism and concurrency.

Busa: Parallelism means executing multiple tasks at the exact same time using multiple cores, while concurrency means managing multiple tasks that can make progress during the same period, even if they take turns running.

Geronimo: Concurrency features overlapping tasks or program copies that communicate. Parallelism runs multiple program copies simultaneously on different data, generally incurring higher time and memory overhead.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Code:

```cpp
#include <iostream>
#include <thread>

//1. Create a global variable of type integer
int global_var = 0;

//2. Create an add function that takes an integer parameter and adds to the global
void add(int val) {
    global_var += val;
}

int main() {
    //3. Use multi-threading techniques to create 3 threads and pass the add functi
    std::thread t1(add, 5);
    std::thread t2(add, 10);
    std::thread t3(add, 15);

    //4. Display value at different combinations of using join() per thread
    t1.join();
    std::cout << "After T1.join(), Global Variable: " << global_var << std::endl;

    t2.join();
    std::cout << "After T2.join(), Global Variable: " << global_var << std::endl;

    t3.join();
    std::cout << "After T3.join(), Global Variable: " << global_var << std::endl;

    return 0;
}
```

Output:

```
After T1.join(), Global Variable: 30
After T2.join(), Global Variable: 30
After T3.join(), Global Variable: 30
```

Analysis:

Spawning 3 threads to update a shared global variable concurrently demonstrates basic multithreading execution. While sequential “join()” calls ensure the orderly completion, modifying the shared memory without sync tools like mutexes can introduce racing conditions and undefined behavior.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code:

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <algorithm>

//Standard merge function for Merge Sort
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

//Sequential Merge Sort
void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```cpp
//Parallel Merge Sort wrapper utilizing a thread for one half
void parallelMergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        // Limit depth to prevent excessive thread creation overhead
        if (depth < 2) {
            std::thread t(parallelMergeSort, std::ref(arr), left, mid, depth + 1);
            parallelMergeSort(arr, mid + 1, right, depth + 1);
            t.join();
        } else {
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
        }
        merge(arr, left, mid, right);
    }
}

int main() {
    std::vector<int> data = {38, 27, 43, 3, 9, 82, 10};

    std::cout << "Original array: ";
    for (int val : data) std::cout << val << " ";
    std::cout << std::endl;

    parallelMergeSort(data, 0, data.size() - 1);

    std::cout << "Sorted array using Multi-threaded Merge Sort: ";
    for (int val : data) std::cout << val << " ";
    std::cout << std::endl;

    return 0;
}
```

Output:

```
Original array: 38 27 43 3 9 82 10
Sorted array using Multi-threaded Merge Sort: 3 9 10 27 38 43 82
PS C:\Users\TIPQC\Documents\CPE010_GERONIMO>
```

Analysis:

Applying multithreading to Merge Sort accelerates the performance by sorting the independent sub-arrays simultaneously across multiple cores.Though, limiting its recursion depth is needed to control the thread creation overhead to prevent performance drops from excessive context switching.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.